VIT-AP UNIVERSITY

Course code : **CSE3009**
Course title : **No SQL Data Bases**
Module : **1**
Topic : **5**

Speeding performance by strategic use of RAM, SSD, and disk

Objectives

This session will give the knowledge about

- Speeding performance by strategic use of RAM, SSD, and disk
- How do NoSQL systems use different types of memory to increase system performance?

Role of a RAM in Databases

We know that when we turn off their computer after a long work day, the data in RAM is erased and must be reloaded.

Data on solid state drives (SSDs) and hard disk drives (HDDs) persists.

We also know that RAM access is fast than the disk access.

The time required to access the SSDs and HDDs are higher than the time required by the RAM to do some hashing calculations.

Role of a RAM in Databases

Solution to Faster Access

The solution to faster systems is to keep as much of the right information in RAM as you can, and check your local servers to see if they might also have a copy.

This local fast data store is often called a RAM cache or memory cache.

Many memory caches use a simple timestamp for each block of memory in the cache as a way of keeping the most recently used objects in memory.

When memory fills up, the timestamp is used to determine which items in memory are the oldest and should be overwritten.

Speeding performance in NoSQL

The effective use of RAM cache is predicated on the efficient answer to the question,

- “Have we run this query before?” or equivalently,
- “Have we seen this document before?”

These questions can be answered by using consistent hashing, which lets you know if an item is already in the cache or if you need to retrieve it from SSD or HDD.

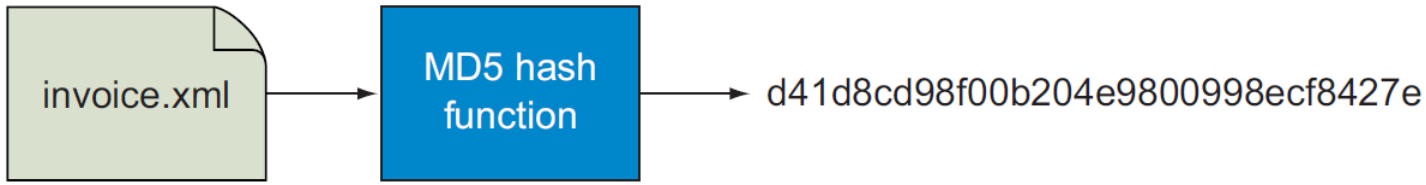
Consistent hashing

NoSQL systems expand on this concept and use a technique called consistent hashing to keep the most frequently used data in your cache.

Consistent hashing quickly tells you if a new query or document is the same as one already in your cache.

Knowing this information prevents you from making unnecessary calls to disk for information and keeps your databases running fast.

Consistent hashing – Process



let $hash := hash($invoice, 'md5')

A hash string (also known as a checksum or hash) is a process that calculates a sequence of letters by looking at each byte of a document.

The hash string uniquely identifies each document and can be used to determine whether the document you're presented with is the same document you already have on hand.

If there's any difference between two documents (even a single byte), the resulting hash will be different.

Hash collisions

There’s an infinitesimally small chance that two different documents could generate the same hash value, resulting in a hash collision.

Systems that use hashes for security verification, like government or high-security systems, require hash values to be greater than 128 bits.

In these situations, algorithms that generate a hash value greater than 128 bits like SHA-1, SHA-256, SHA-384, or SHA-512 are preferred.

Consistent hashing - Advantages

Hash values can be created for simple queries or complex JSON or XML documents.

Once you have your hash value, you can use it to compare sending and receiving.

Consistent hashing occurs when two different processes running on different nodes in your network create the same hash for the same object.

Consistent hashing confirms that the information in the document hasn’t been altered and allows you to determine whether the object exists in your cache or message store

Conclusion

Consistent hashing is an important tool to keep your cache current and your system running fast, even when caches are spread over many distributed systems.

Consistent hashing can also be used to assign documents to specific database nodes on distributed systems and to quickly compare remote databases when they need to be synchronized.

Distributed NoSQL systems rely on hashing for rapidly enhancing database read times without getting in the way of write transactions.

Summary

This session will give the knowledge about

- Speeding performance by strategic use of RAM, SSD, and disk
- How do NoSQL systems use different types of memory to increase system performance?